

Three-VM Public-Key Infrastructure Lab

Ubuntu 24.04 LTS - Certificate Authority, HTTPS Server, and Web Client

Purpose

In this lab, you will construct a small public-key infrastructure rather than use a prepared security-lab image. You will install three Ubuntu virtual machines, create a private certificate authority, enroll a web server, deploy an HTTPS site, and evaluate certificate validation from a separate client.

The three machines represent different security roles:

- The **CA** decides whether to bind a public key to an identity.
- The **web server** creates and retains its private key.
- The **client** independently decides which CA roots to trust.

This lab does not include a man-in-the-middle attack.

Learning objectives

After completing the lab, you should be able to:

- 1 Build and validate an isolated three-machine virtual network.
- 2 Explain which PKI artifacts are public and which must remain private.
- 3 Create an OpenSSL CA database and a self-signed root certificate.
- 4 Generate and inspect a server key and certificate signing request (CSR).
- 5 Review and sign a CSR as a CA operator.
- 6 Transfer certificates without transferring private keys.
- 7 Deploy a CA-issued certificate on Apache.
- 8 Explain issuer trust, certificate chains, SAN hostname validation, and certificate validity periods.
- 9 Distinguish explicit application trust from browser trust.

Rules and safety boundaries

The personalized name used in this exercise is a local laboratory override. It does not create or modify Penn State DNS.

- Do not expose any lab VM or Apache service to the public Internet.
- Do not request a publicly trusted certificate for the lab hostname.
- Do not install the lab root CA on your everyday host operating system or personal browser profile.
- Trust the lab CA only in the dedicated client VM.

- Never transfer, submit, or take screenshots of `ca.key` or `server.key`.
- Disconnect NAT before beginning the certificate exercises.

Required host resources

- Windows or macOS host
- VMware Workstation on Windows or VMware Fusion on macOS
- 16 GB host memory
- 50 GB free disk space
- Hardware virtualization enabled
- Ubuntu 24.04 LTS installation media for the host CPU architecture

Use AMD64 installation media on Intel/AMD systems. Use ARM64 installation media on Apple Silicon. Do not attempt to boot an AMD64 image on Apple Silicon.

Address and identity plan

Replace `abc1234` everywhere with the portion of your Penn State email address before `@psu.edu`. Convert it to lowercase.

Item	Example value
Penn State email	<code>abc1234@psu.edu</code>
Student ID	<code>abc1234</code>
HTTPS hostname	<code>abc1234.psu.edu</code>
CA VM hostname	<code>pki-ca</code>
Server VM hostname	<code>pki-server</code>
Client VM hostname	<code>pki-client</code>

Network assignments:

Role	Address	Gateway	DNS
CA	<code>10.77.0.10/24</code>	None	None
Server	<code>10.77.0.20/24</code>	None	None
Client	<code>10.77.0.30/24</code>	None	None

Artifact custody model

Artifact	Created on	May leave its origin?	Purpose
ca.key	CA	No	Signs certificates
ca.crt	CA	Yes	Establishes client trust
CA database	CA	No	Records issued certificates
server.key	Server	No	Proves server identity
server.csr	Server	Yes	Requests a certificate
server.crt	CA	Yes	Binds the server key to its name

Private keys never cross a machine boundary.

Report and evidence guidance

Document every numbered section (Sections 1 through 7) in your lab report. For each section:

- 1 Provide an annotated screenshot that identifies the key result or evidence produced in that section.
- 2 Answer every question posed in that section.
- 3 Explain unexpected output, warnings, or deviations from the instructions.

One screenshot per numbered section should normally be sufficient. You may provide multiple screenshots when the required details from its subsections cannot be captured legibly in one image. Crop screenshots to the relevant VM and output, and use arrows, boxes, or labels to identify the evidence being graded. A screenshot without annotation is incomplete.

Never expose a passphrase or private-key contents in a screenshot. Command output may be transcribed when it is more readable than an additional image.

Prerequisite and isolation checkpoint

Complete **PKI Lab Environment Build - VMware on Windows or macOS** before beginning this lab. Follow only the host-platform subsections that apply to your computer.

The precursor produces three Ubuntu VMs with two adapters each and a personalized HTTP-only website. Do not repeat the VM, package, networking, or HTTP setup here.

Before Section 1:

- 1 Shut down all three VMs.
- 2 Disconnect Adapter 1, the NAT adapter, on each VM.
- 3 Leave Adapter 2 connected to the PKI-LAB LAN Segment or custom private network.
- 4 Start all three VMs.

Run the matching preflight command on each VM:

```
sudo bash scripts/preflight-lab-vm.sh ca abc1234
sudo bash scripts/preflight-lab-vm.sh server abc1234
sudo bash scripts/preflight-lab-vm.sh client abc1234
```

Run only the command matching that VM and replace the example ID. Confirm that the personalized HTTP page still loads from `pki-client` and that no default IPv4 or IPv6 route exists.

Do not remove the NAT adapters; leave them disconnected throughout the PKI lab. Resolve every preflight failure before starting Section 1.

Phase 1: Certificate enrollment and issuance

Phase 1 includes Sections 1 through 3. The web server first creates and submits its key and CSR. You will then establish the CA and use it to review and issue the server certificate.

Section 1: Create the server key and CSR

Perform this section only on `pki-server`.

1.1 Define the personalized identity

```
STUDENT_ID=abc1234
SERVER_NAME="${STUDENT_ID}.psu.edu"
mkdir -p ~/pki-server/{private,requests,certs}
chmod 700 ~/pki-server/private
```

1.2 Generate the private key

```
openssl genpkey -algorithm RSA \
  -pkeyopt rsa_keygen_bits:2048 \
  -aes-256-cbc \
  -out ~/pki-server/private/server.key
chmod 600 ~/pki-server/private/server.key
```

Choose a server-key passphrase. Do not place it on the command line.

Inspect only the non-secret properties:

```
openssl pkey -in ~/pki-server/private/server.key \
  -pubout -out ~/pki-server/certs/server-public.pem
openssl pkey -pubin -in ~/pki-server/certs/server-public.pem \
  -noout -text
```

1.3 Generate the CSR

```
openssl req -new -sha256 \
  -key ~/pki-server/private/server.key \
  -out ~/pki-server/requests/${SERVER_NAME}.csr \
  -subj "/C=US/ST=Pennsylvania/O=Penn State PKI Lab/OU=Student Web Server/CN=${SERVER_NAME}" \
  -addext "subjectAltName=DNS:${SERVER_NAME}"
```

Inspect and verify it:

```
openssl req -in ~/pki-server/requests/${SERVER_NAME}.csr \
  -noout -subject -text -verify
sha256sum ~/pki-server/requests/${SERVER_NAME}.csr
```

Confirm that the requested SAN is exactly your personalized hostname. Leave the CSR on `pki-server` until the CA workspace exists in Section 2.

Section 2: Become a certificate authority

Perform this part only on `pki-ca`.

2.1 Create the CA workspace

```
mkdir -p ~/pki-lab-ca/ca/{certs,crl,newcerts,private}
mkdir -p ~/pki-lab-ca/{incoming,outgoing,public}
touch ~/pki-lab-ca/ca/index.txt
printf '1000\n' > ~/pki-lab-ca/ca/serial
chmod 700 ~/pki-lab-ca/ca/private
```

Open the CA configuration file in nano:

```
nano ~/pki-lab-ca/openssl-ca.cnf
```

Enter the following content. The CA itself is shared by the three roles and is not named after the website.

```
[ ca ]
default_ca = CA_default

[ CA_default ]
dir                = ./ca
certs              = $dir/certs
crl_dir            = $dir/crl
database           = $dir/index.txt
new_certs_dir      = $dir/newcerts
certificate        = $dir/certs/ca.crt
serial             = $dir/serial
private_key        = $dir/private/ca.key
default_days       = 365
default_md         = sha256
policy             = policy_server
email_in_dn        = no
unique_subject     = no
copy_extensions    = none
x509_extensions    = server_cert

[ policy_server ]
countryName        = supplied
stateOrProvinceName = supplied
organizationName   = supplied
organizationalUnitName = optional
commonName         = supplied
emailAddress       = optional

[ req ]
default_bits       = 4096
default_md         = sha256
distinguished_name = ca_dn
x509_extensions    = root_ca
prompt             = no

[ ca_dn ]
C = US
ST = Pennsylvania
O = Penn State PKI Lab
OU = Laboratory Certificate Authority
CN = Penn State PKI Lab Root CA

[ root_ca ]
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid:always,issuer
basicConstraints      = critical,CA:true,pathlen:0
keyUsage              = critical,keyCertSign,cRLSign

[ server_cert ]
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid,issuer
basicConstraints      = critical,CA:false
keyUsage              = critical,digitalSignature,keyEncipherment
extendedKeyUsage       = serverAuth
```

Save the file by pressing **Ctrl+O**, press **Enter** to confirm the filename, and then press **Ctrl+X** to exit nano.

2.2 Generate the root key and certificate

Change to the CA workspace so its relative database paths resolve correctly:

```
cd ~/pki-lab-ca
openssl req -config openssl-ca.cnf \
    -new -x509 -newkey rsa:4096 -sha256 -days 3650 \
    -keyout ca/private/ca.key \
    -out ca/certs/ca.crt
chmod 600 ca/private/ca.key
cp ca/certs/ca.crt public/ca.crt
chmod 644 ca/certs/ca.crt public/ca.crt
```

Choose a strong CA passphrase. Do not place it on the command line, in a script, or in your report.

2.3 Inspect the root

```
openssl x509 -in ca/certs/ca.crt -noout \
  -subject -issuer -serial -dates -fingerprint -sha256
openssl x509 -in ca/certs/ca.crt -noout -text
openssl verify -CAfile ca/certs/ca.crt ca/certs/ca.crt
```

Answer:

- 1 Which fields show that the certificate is self-signed?
- 2 Which extension authorizes this certificate to act as a CA?
- 3 What operations does its Key Usage permit?
- 4 Why is the root certificate public while the root key is private?

Do not display or capture the private RSA parameters in a screenshot.

Section 3: Review and issue the server certificate

3.1 Transfer only the CSR

On `pki-server`:

```
STUDENT_ID=abc1234
SERVER_NAME="${STUDENT_ID}.psu.edu"
scp ~/pki-server/requests/${SERVER_NAME}.csr \
  YOUR_CA_USERNAME@pki-ca:~/pki-lab-ca/incoming/
```

Verify the CA's SSH host-key fingerprint when prompted. Do not disable SSH host-key checking. Do not transfer `server.key`.

Return to `pki-ca`.

3.2 Review the request

```
STUDENT_ID=abc1234
SERVER_NAME="${STUDENT_ID}.psu.edu"
cd ~/pki-lab-ca

openssl req -in incoming/${SERVER_NAME}.csr \
  -noout -subject -text -verify
sha256sum incoming/${SERVER_NAME}.csr
```

Before signing, verify:

- The CSR signature is valid.
- The Common Name is `${STUDENT_ID}.psu.edu`.
- The SAN contains exactly `DNS:${STUDENT_ID}.psu.edu`.
- The public key is RSA and at least 2048 bits.
- The request contains no unexpected extensions.

3.3 Create a controlled extension file

The CA must authorize the SAN rather than blindly copy arbitrary CSR extensions. Open `server-ext.cnf`:

```
nano server-ext.cnf
```

Replace `abc1234` with your student ID, then enter:

```
[ server_cert ]
```

```
subjectKeyIdentifier    = hash
authorityKeyIdentifier  = keyid,issuer
basicConstraints         = critical,CA:false
keyUsage                = critical,digitalSignature,keyEncipherment
extendedKeyUsage        = serverAuth
subjectAltName          = DNS:abc1234.psu.edu
```

Press **Ctrl+O**, **Enter**, and **Ctrl+X** to save and exit.

3.4 Sign the request

```
openssl ca -config openssl-ca.cnf \
  -extensions server_cert \
  -extfile server-ext.cnf \
  -days 365 -md sha256 \
  -in incoming/${SERVER_NAME}.csr \
  -out outgoing/${SERVER_NAME}.cert
```

Read OpenSSL's request summary before approving issuance. Enter the CA passphrase only when prompted.

3.5 Inspect the issued certificate and CA database

```
openssl x509 -in outgoing/${SERVER_NAME}.cert -noout \
  -subject -issuer -serial -dates -fingerprint -sha256
openssl x509 -in outgoing/${SERVER_NAME}.cert -noout -text
openssl verify -CAfile public/ca.cert outgoing/${SERVER_NAME}.cert
cat ca/index.txt
cat ca/serial
```

Answer:

- 1 How does the certificate serial relate to the CA database?
- 2 Why is CA:FALSE critical for a server certificate?
- 3 Why did the CA construct the SAN extension instead of copying every requested CSR extension?

Phase 2: Certificate deployment

Phase 2 includes Sections 4 and 5. You will retrieve the issued certificate, verify it against the retained server key, and convert the precursor's HTTP server into an HTTPS-capable Apache site.

Section 4: Retrieve and verify the certificate

Return to `pki-server`.

```
STUDENT_ID=abc1234
SERVER_NAME="${STUDENT_ID}.psu.edu"
```

```
scp YOUR_CA_USERNAME@pki-ca:~/pki-lab-ca/outgoing/${SERVER_NAME}.cert \
~/pki-server/certs/server.crt
scp YOUR_CA_USERNAME@pki-ca:~/pki-lab-ca/public/ca.crt \
~/pki-server/certs/ca.crt
```

Verify the chain and hostname:

```
openssl verify -CAfile ~/pki-server/certs/ca.crt \
-verify_hostname "$SERVER_NAME" \
~/pki-server/certs/server.crt
```

Verify that the certificate matches the retained private key:

```
openssl x509 -in ~/pki-server/certs/server.crt -pubkey -noout |
openssl pkey -pubin -outform DER |
sha256sum

openssl pkey -in ~/pki-server/private/server.key -pubout -outform DER |
sha256sum
```

The two SHA-256 values must match.

Section 5: Deploy the certificate on Apache

5.1 Create a deployment copy of the key

Apache must start unattended under `systemd`. Preserve the encrypted original and create a root-readable, unencrypted deployment copy:

```
sudo install -d -o root -g root -m 700 /etc/apache2/pki-lab
openssl pkey -in ~/pki-server/private/server.key \
-out ~/pki-server/private/server-deploy.key
chmod 600 ~/pki-server/private/server-deploy.key
sudo install -o root -g root -m 600 ~/pki-server/private/server-deploy.key \
/etc/apache2/pki-lab/server.key
rm -f ~/pki-server/private/server-deploy.key

sudo install -o root -g root -m 644 ~/pki-server/certs/server.crt \
/etc/apache2/pki-lab/server.crt
```

Explain why the CA key remains encrypted while Apache uses a protected, unencrypted deployment copy of the server key.

5.2 Confirm the personalized web content

The precursor created the document root and HTTP page. Confirm that they are still present instead of rebuilding them:

```
test -f "/var/www/${SERVER_NAME}/index.html"
grep -F "$STUDENT_ID" "/var/www/${SERVER_NAME}/index.html"
curl -H "Host: ${SERVER_NAME}" http://127.0.0.1/
```

All three commands must succeed, and the page must visibly contain your user ID. The HTTPS virtual host will reuse this existing document root.

5.3 Configure the HTTPS virtual host

```
sudo nano /etc/apache2/sites-available/pki-lab.conf
```

Replace every occurrence of abc1234 with your student ID, then enter:

```
<VirtualHost *:443>
    ServerName abc1234.psu.edu
    DocumentRoot /var/www/abc1234.psu.edu
    DirectoryIndex index.html

    SSLEngine on
    SSLCertificateFile /etc/apache2/pki-lab/server.crt
    SSLCertificateKeyFile /etc/apache2/pki-lab/server.key

    ErrorLog ${APACHE_LOG_DIR}/pki-lab-error.log
    CustomLog ${APACHE_LOG_DIR}/pki-lab-access.log combined

    <Directory /var/www/abc1234.psu.edu>
        Require all granted
    </Directory>
</VirtualHost>
```

Press **Ctrl+O**, **Enter**, and **Ctrl+X** to save and exit. Then enable the HTTPS site:

```
sudo a2enmod ssl
sudo a2dissite 000-default default-ssl 2>/dev/null || true
sudo a2ensite pki-lab
sudo apache2ctl configtest
sudo systemctl restart apache2
sudo systemctl --no-pager --full status apache2
```

Confirm that Apache listens on TCP port 443:

```
sudo ss -lntp | grep ':443'
```

Phase 3: Client testing and validation

Phase 3 includes Sections 6 and 7. You will test explicit and browser trust, inspect the TLS service, and demonstrate that trust and hostname authorization are independent validation requirements.

Section 6: Evaluate the site from the client

Perform this part only on `pki-client`.

6.1 Obtain the public root certificate

```
mkdir -p ~/pki-client
scp YOUR_CA_USERNAME@pki-ca:~/pki-lab-ca/public/ca.crt \
~/pki-client/ca.crt
chmod 644 ~/pki-client/ca.crt
```

Compare its SHA-256 fingerprint with the fingerprint recorded directly on the CA:

```
openssl x509 -in ~/pki-client/ca.crt -noout -fingerprint -sha256
```

Receiving a file and establishing that it is the intended trust anchor are separate decisions.

6.2 Test without trust

```
STUDENT_ID=abc1234
SERVER_NAME="${STUDENT_ID}.psu.edu"
```

```
curl -v "https://${SERVER_NAME}/"
```

The request should fail because the lab root is not in curl's default trust store. Record the error without bypassing verification.

Do not use `curl -k` or `--insecure` as a solution.

6.3 Test with explicit trust

```
curl --cacert ~/pki-client/ca.crt "https://${SERVER_NAME}/"
```

This request should succeed. The root has been trusted for this invocation only; it has not been installed system-wide.

Inspect the TLS service independently:

```
openssl s_client \
  -connect ${SERVER_NAME}:443 \
  -servername ${SERVER_NAME} \
  -CAfile ~/pki-client/ca.crt \
  -verify_hostname ${SERVER_NAME} \
  -verify_return_error </dev/null
```

Record the negotiated TLS version, cipher, peer subject, issuer, and final verification result.

6.4 Test Firefox before trust

Create a dedicated Firefox profile named PKI - Lab. Do not reuse a personal profile. Visit:

```
https://abc1234.psu.edu/
```

Replace the example ID. Record the certificate warning and identify the unknown issuer as the cause. Do not permanently override the warning.

6.5 Import the root into the lab profile

In Firefox's certificate settings, import `~/pki-client/ca.crt` under Authorities and permit it to identify websites. The exact menu wording may vary with the installed Firefox release.

Close and reopen the lab profile, then revisit the personalized URL. Inspect the connection and certificate details. Record:

- Browser-visible subject
- Issuer
- SAN
- Validity period
- SHA-256 fingerprint

Section 7: Test validation boundaries

7.1 Hostname mismatch by IP address

From the client:

```
curl --cacert ~/pki-client/ca.crt https://10.77.0.20/
```

Explain why a trusted issuer does not make this connection valid.

7.2 Hostname mismatch by an unlisted name

Temporarily add this entry to the client's `/etc/hosts`:

```
10.77.0.20 wrong-name.psu.edu
```

Then run:

```
curl --cacert ~/pki-client/ca.crt https://wrong-name.psu.edu/
```

Remove the temporary entry after recording the result.

7.3 Remove browser trust

Remove the Penn State PKI Lab Root CA from the dedicated Firefox profile and revisit the correct URL. The unknown-issuer failure should return.

Summarize the outcomes:

Test	Issuer trusted?	Name authorized?	Expected outcome
Correct name before import	No	Yes	Reject
Correct name with <code>--cacert</code>	Yes	Yes	Accept
Correct name after Firefox import	Yes	Yes	Accept
Server IP address	Yes	No	Reject
Unlisted hostname	Yes	No	Reject
Correct name after trust removal	No	Yes	Reject

Report content checklist

Use the per-section evidence guidance near the beginning of this manual. Taken together, the annotated evidence and written answers should demonstrate:

- VM specifications, architecture, and Ubuntu version
- Final isolated networking and passing preflight results

- Proper custody of private and public PKI artifacts
- Root certificate, CSR, and issued certificate inspection
- CA request review and issuance records
- A match between the certificate public key and server private key
- A valid Apache configuration and working HTTPS service
- Command-line and Firefox trust results
- Completed validation-boundary results with explanations
- Final reflection

Do not include:

- `ca . key`
- `server . key`
- Passphrases
- Screenshots displaying private-key contents
- Whole VM images

Cleanup guidance

After your report has been completed and accepted, shut down and delete all three lab VMs and their associated virtual disks and snapshots. Reusing these VMs for other work is not recommended because the client trusted a private laboratory CA and the server and CA contain sensitive private keys. Verify that you are deleting only the three lab VMs before confirming the destructive operation in VMware.

Final reflection: explain why successful HTTPS authentication required all of the following at once: custody of the server private key, a CA signature, an authorized SAN, a currently valid certificate, and client trust in the CA.

Acknowledgment

This exercise adapts the certificate-authority, server-enrollment, Apache, and client-trust sequence from Wenliang Du's SEED Public-Key Infrastructure Lab. It uses a new three-VM, non-containerized environment and does not include the SEED lab's MITM activities.